

1 Một giai đoạn và cuộc đánh đổi tốc độ (One-Stage Detection and the Speed Trade-off)

PHỤ THUỘC C/14-bai-toan-cv-loi/01-detection-hai-giai-doan (anchor, NMS, RoI); B/09-thiet-ke-ham-mat-mat (focal loss, mất cân bằng lớp); C/13/02-tu-chuoi-toi-attention (attention, cần cho phần DETR).

TÓM TẮT

Mục này trả lời ba câu hỏi. Vì sao bỏ giai đoạn đề xuất vùng lại làm lộ ra mất cân bằng nền–vật thể? Vì sao vấn đề đó được chữa ở tầng hàm mất mát chứ không ở tầng kiến trúc? Và vì sao chuỗi ATSS → SimOTA → Hungarian matching mới là trực tiến hoá thật của detection hiện đại? Đọc xong, trong bất kỳ bảng so sánh detector nào bạn cũng tách được phần chênh lệch *do kiến trúc* khỏi phần chênh lệch *do gán mẫu và công thức huấn luyện*. Bạn cũng biết tìm thông tin đó ở đâu khi paper không nói. Nếu chỉ nhớ một điều: cách gán mẫu cho 10^5 vị trí dự đoán là một phần của *định nghĩa bài toán tối ưu*. Hai detector gán mẫu khác nhau vì thế là hai bài toán khác nhau. So sánh chúng rồi quy kết cho kiến trúc là sai ngay từ tiền đề.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — một số duy nhất đo mức sai của dự đoán so với nhãn; toàn bộ việc huấn luyện là chỉnh tham số để số đó nhỏ đi. Đổi hàm mất mát là đổi *mục tiêu*, không phải đổi mô hình.
- **cross-entropy** — hàm mất mát chuẩn cho phân loại: $-\log p_t$ với p_t là xác suất mô hình gán cho lớp đúng. Đoán đúng chắc chắn thì mất mát gần 0; đoán sai chắc chắn thì mất mát rất lớn.
- **mẫu dương / mẫu âm** — với mỗi vị trí dự đoán, “dương” nghĩa là vị trí đó được coi là chứa vật thể và phải học nhãn của vật đó; “âm” nghĩa là nền. Ai được coi là dương là do bước gán mẫu quyết định.
- **gán mẫu (label assignment)** — quy tắc quyết định mỗi trong hàng trăm nghìn vị trí dự đoán phải học điều gì. Nó không phải một khối tính toán mà là một phần của *định nghĩa* hàm mất mát — đây là ý trung tâm của cả mục.
- **anchor và anchor-free** — có anchor là dự đoán độ lệch so với các khung mẫu định sẵn; anchor-free là dự đoán trực tiếp từ mỗi ô lưới, chẳng hạn bốn khoảng cách tới bốn cạnh của khung.
- **NMS** — hậu xử lý xoá khung trùng: giữ khung điểm cao nhất rồi xoá mọi khung chồng lấn với nó quá ngưỡng IoU. Vì nó xét quan hệ *giữa* các dự đoán nên không thể huấn luyện cùng phần còn lại.
- **truy vấn (query) trong DETR** — một vector học được đóng vai “một chỗ trống để điền một vật thể”. Mô hình có N truy vấn cố định, mỗi truy vấn tự đi hỏi feature map rồi trả về một khung cộng một nhãn.
- **epoch** — một lượt đi hết toàn bộ tập huấn luyện. “Hội tụ sau 500 epoch” nghĩa là phải quét dữ liệu 500 lần mới đạt kết quả tốt — rất đắt.

Đặt vấn đề

Giai đoạn đề xuất vùng trong Faster R-CNN giải quyết được một vấn đề rất cụ thể: nó *lọc* hàng chục nghìn vị trí ứng viên xuống còn vài trăm, và mọi thứ phía sau chỉ phải làm việc trên một tập nhỏ, đã tương đối cân bằng giữa mẫu dương và mẫu âm. Cái giá là một vòng lặp nữa qua bộ nhớ và một lượng tính toán tỉ lệ với số đề xuất. Với robot phải chạy 30 khung hình mỗi giây trên một máy nhúng, cái giá đó thường không trả nổi. Câu hỏi tự nhiên là: bỏ hẳn giai đoạn đề xuất, cho mạng dự đoán thẳng trên lưới đặc trưng, thì mất gì?

Học xong mục này bạn làm được gì mà trước đó không làm được? Đọc phần thực nghiệm của một paper detection, bạn biết ngay cần kiểm tra ba dòng nào trong code trước khi tin bảng kết quả.

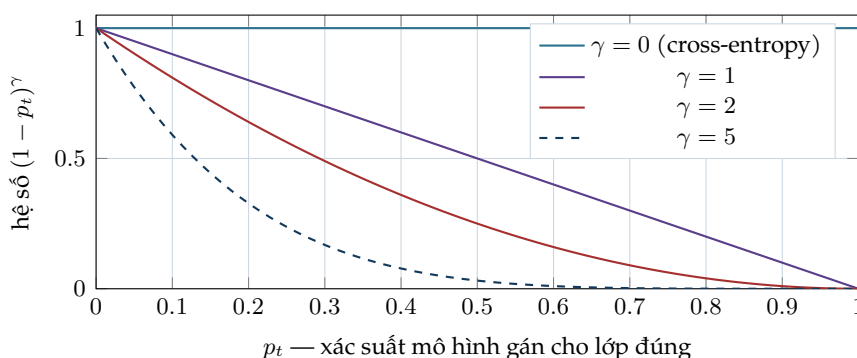
YOLO và SSD trả lời bằng cách làm đúng thế [1], [2]. Kết quả là tốc độ tăng vọt, nhưng một vấn đề mới xuất hiện mà kiến trúc hai giai đoạn chưa từng phải đối mặt, bởi vì bộ lọc đề xuất đã âm thầm che nó đi: **mất cân bằng nền-vật thể** (*foreground-background imbalance*). Bài học lớn nhất của mục này nằm ở chỗ vấn đề đó được giải quyết ở tầng **hàm mất mát** (*loss function*), không phải ở tầng kiến trúc. Câu chuyện sau đó còn cho thấy một điều nữa: phần lớn khoảng cách giữa các detector hiện đại nằm ở những chi tiết *không được viết trong abstract*.

A. Mất cân bằng, và vì sao nó là vấn đề của loss (Class Imbalance)

Hãy đếm. Một mạng một giai đoạn với **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) năm tầng, ảnh 800×1333 , chín **anchor** mỗi vị trí, sinh ra cỡ 10^5 dự đoán mỗi ảnh. Một ảnh COCO trung bình có bảy vật thể; kể cả khi mỗi vật khớp vài anchor thì số mẫu dương vẫn chỉ vài chục. Tỷ lệ âm trên dương do đó vào khoảng 10^3 tới 10^4 .

Con số đó chưa đủ để thấy vấn đề. Cái đáng sợ nằm ở tổng gradient. Giả sử sau vài epoch, mạng đã học được rằng gần như mọi thứ là nền. Mỗi anchor nền khi đó có xác suất đúng $p_t = 0,99$, đóng góp **entropy chéo** (*cross-entropy*) $-\log 0,99 \approx 0,01$. Nghe như không đáng kể — nhưng 10^5 anchor nền cho tổng $10^5 \times 0,01 = 1000$. Trong khi đó ba mươi anchor dương khó, mỗi cái $p_t = 0,5$ và loss 0,69, chỉ cho tổng 21. Tín hiệu học thật bị nhấn chìm dưới tỷ lệ 50:1 bởi những mẫu mà mạng đã làm đúng rồi. Đây là ví dụ nhỏ nhất giải thích vì sao “hạ learning rate” hay “train lâu hơn” không cứu được: bản thân hàm mất mát đang chỉ sai hướng.

Cách chữa cổ điển là **hard negative mining** — SSD giữ tỷ lệ âm/dương 3:1 bằng cách chỉ lấy các mẫu âm có loss cao nhất. Cách này hoạt động nhưng đưa thêm một heuristic rời rạc, không khả vi vào giữa vòng huấn luyện. **Focal loss** thay nó bằng một hệ số điều biến trơn: $FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t$ [3]. Với $\gamma = 2$, mẫu dễ $p_t = 0,99$ bị nhân với $(0,01)^2 = 10^{-4}$. Mẫu khó $p_t = 0,5$ chỉ bị nhân 0,25. Lập lại phép đếm ở trên: tổng đóng góp của nền tụt từ 1000 xuống 0,1, trong khi mẫu dương khó chỉ giảm bốn lần. Tỷ lệ đảo ngược hoàn toàn.



Hình 1. Hệ số điều biến của focal loss. Điều quan trọng không phải mẫu khó được đề cao, mà là mẫu *đã đúng* bị hạ xuống gần như bằng không — với $\gamma = 2$, một anchor nền chắc chắn đóng góp ít hơn bốn bậc so với cross-entropy. Vì γ chỉ nén chứ không đảo thứ tự, nó không thể sửa được một tập nhãn sai.

Hình 1 cũng cho thấy giới hạn. Hệ số luôn dương và đơn điệu. Nên khi một mẫu bị gán nhãn sai, mô hình dự đoán *đúng theo thực tế nhưng sai theo nhãn*, focal loss coi nó là mẫu khó và *tăng* trọng số cho nó. Trên dữ liệu nhiễu nhãn, đây là chế độ hỏng thật, không phải lý thuyết.

↪ **trường hợp riêng của B/09** — focal loss là một ca cụ thể của nguyên tắc tổng quát “sửa mất cân bằng ở tầng hàm mất mát thay vì ở tầng lấy mẫu dữ liệu”.

Kết luận rút ra từ RetinaNet vượt xa bản thân hàm loss. Suốt vài năm, khoảng cách giữa một giai đoạn và hai giai đoạn được giải thích bằng “kiến trúc hai giai đoạn tinh vi hơn”. Hoá ra phần lớn khoảng cách đó là hệ quả của việc giai đoạn đề xuất đã ngầm cân bằng lại tập mẫu. Sửa ở tầng loss là đủ.

B. Bỏ anchor, và câu hỏi thật sự là gán mẫu (Label Assignment)

TRỰC GIÁC

Kiến trúc là thí sinh; cách gán mẫu là *đáp án*. Một detector một giai đoạn có 10^5 ô trên lưới, và gán mẫu chính là việc điền vào đáp án cho từng ô: ô này phải nói “có xe ở đây”, ô kia phải nói “không”. Đối cách gán mẫu không phải là đối thí sinh, mà là đối đáp án — tức đối hần bài toán tối ưu. Vì thế so hai detector dùng hai cách gán mẫu khác nhau rồi kết luận về kiến trúc cũng vô nghĩa như so điểm hai thí sinh làm hai đề khác nhau.

Mạch 2 — một giai đoạn, từ anchor tới tập hợp

- 1 Vấn đề** anchor là một siêu tham số thủ công mang theo thống kê của tập dữ liệu.

Giải pháp họ *anchor-free* dự đoán trực tiếp từ mỗi vị trí trên lưới: FCOS dự đoán bốn khoảng cách từ điểm đó tới bốn cạnh của khung [4], CenterNet dự đoán một heatmap tâm vật cộng kích thước [5].

Giá phải trả không còn tập anchor, nhưng bây giờ phải trả lời một câu hỏi mới và khó hơn: *vị trí nào trên lưới chịu trách nhiệm cho vật nào*. Bài toán không biến mất, nó đổi tên thành gán mẫu.
- 2 Vấn đề** gán mẫu theo ngưỡng IoU cố định là một lựa chọn tùy tiện. Ngưỡng 0,5 khiến vật nhỏ chỉ nhận được một hai mẫu dương trong khi vật lớn nhận hàng chục, tức mô hình được huấn luyện với mức giám sát rất khác nhau tùy kích thước.

Giải pháp ATSS — viết tắt của *adaptive training sample selection* — chọn ngưỡng thích nghi cho từng vật: lấy k ứng viên gần tâm nhất ở mỗi tầng, tính trung bình và độ lệch chuẩn IoU của chúng, dùng $\mu + \sigma$ làm ngưỡng [6].

Hệ quả bài báo cho thấy sau khi thống nhất cách gán mẫu, khác biệt giữa detector có anchor và không anchor gần như biến mất — một trong những kết quả ablation quan trọng nhất của cả lĩnh vực.
- 3 Vấn đề** gán mẫu từng vật độc lập là tối ưu cục bộ. Khi hai vật chồng lấn, một vị trí lưới có thể tốt cho cả hai, và quyết định tham lam cho từng vật riêng lẻ dẫn tới xung đột.

Giải pháp *optimal transport assignment* (OTA) và bản rút gọn SimOTA phát biểu việc gán mẫu thành một bài toán vận tải tối ưu: mỗi ground truth là một nguồn cung k nhãn dương, mỗi vị trí lưới là một điểm cầu, chi phí là tổ hợp loss phân loại và loss hồi quy [7].

Giá phải trả thêm chi phí tính toán trong lúc huấn luyện, không thêm gì lúc suy luận.
- 4 Vấn đề** NMS là heuristic cuối cùng, và nó là heuristic duy nhất thao tác trên quan hệ giữa các dự đoán. Soft-NMS làm mềm bằng cách giảm điểm thay vì xóa hẳn [8], nhưng vẫn là hậu xử lý.

Giải pháp DETR đổi phát biểu: dự đoán một tập hợp cố định N truy vấn, ghép chúng với **chân trị** (ground truth) bằng **ghép cặp Hungary** (Hungarian matching) một-một, và loss chỉ được định nghĩa sau khi ghép [9]. Đây là ý tưởng **dự đoán tập hợp** (set prediction) mà Mục 3 sẽ mượn nguyên vẹn sang segmentation. Vì mỗi vật chỉ có đúng một dự đoán được thưởng, mô hình học cách tự triệt tiêu trùng lặp; NMS biến mất khỏi pipeline.

Giá phải trả hội tụ cực chậm — bản gốc cần khoảng 500 epoch trong khi Faster R-CNN quen dùng lịch 12–36 epoch — và kém trên vật nhỏ.

5 Vấn đề

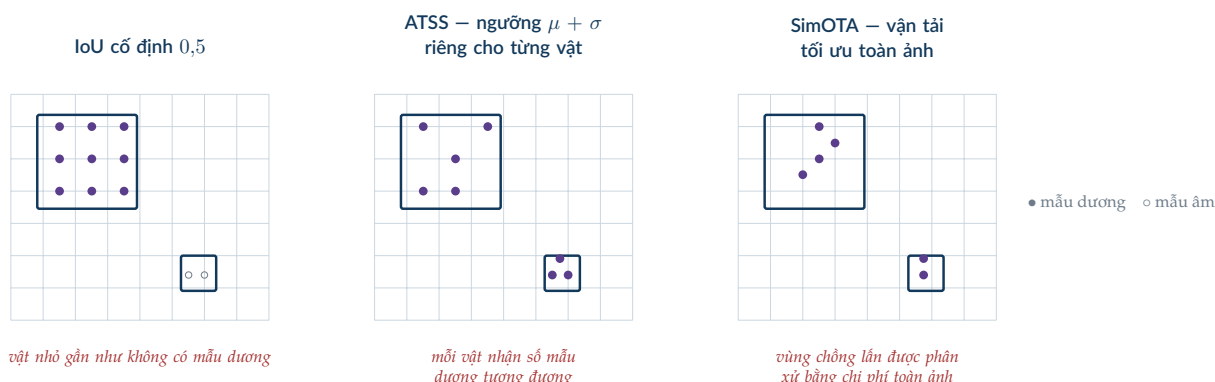
vì sao DETR chậm hội tụ? Hai nguyên nhân được các bản sửa chỉ ra: **attention** dày đặc trên toàn feature map khởi đầu gần như đồng đều nên mất rất lâu để tập trung, và phép ghép một-một dao động giữa các epoch nên tín hiệu giám sát cho mỗi truy vấn không ổn định.

Giải pháp

Deformable DETR cho mỗi truy vấn chỉ lấy mẫu một số ít điểm quanh một điểm tham chiếu học được, giảm cả chi phí lẫn thời gian hội tụ xuống cỡ 50 epoch [10], [11]; DINO thêm truy vấn khởi nhiễu (cho mạng các khung ground truth đã bị nhiễu và bất khôi phục) để ổn định hoá tín hiệu [12].

Hệ quả

tới 2026, dòng thực dụng như RT-DETR và RF-DETR đã đạt được cả tốc độ lẫn độ chính xác mà không cần NMS.



Hình 2. Cùng một ảnh, cùng một mạng, cùng một tập vị trí dự đoán — ba quy tắc gán mẫu cho ba tập mẫu dương khác nhau, và do đó ba hàm mất mát khác nhau. Khung đậm là hai vật thật, một lớn một nhỏ. Với ngưỡng IoU cố định, vật lớn nuốt gần hết ngân sách giám sát còn vật nhỏ hầu như không được dạy; ATSS cân lại bằng ngưỡng tính riêng cho từng vật; SimOTA thêm một bước phân xử toàn ảnh nên vị trí tranh chấp không bị gán cho cả hai. Vị trí các chấm ở đây là minh hoạ nguyên lý, không phải kết quả chạy thật.

Hình 2 cho thấy thứ mà một bảng số không cho thấy: cái thay đổi giữa ba cột không phải mô hình mà là *đề bài* đưa cho mô hình.

Bảng 1. Bốn cách trả lời câu hỏi “vị trí nào chịu trách nhiệm cho vật nào”. Đây là thành phần hiểm khi nằm trong abstract nhưng thường giải thích phần lớn chênh lệch giữa hai detector.

Cách gán	Cơ chế	Giả định	Hỏng khi nào
IoU cố định	Anchor nào có IoU vượt ngưỡng thì là dương	Ngưỡng tốt cho mọi kích thước và tỉ lệ	Vật nhỏ nhận rất ít mẫu dương; đối phân bố kích thước là phải chỉnh lại tay
Centerness / FCOS	Điểm trong vùng trung tâm của khung là dương, có trọng số theo độ lệch tâm	Tâm hình học là điểm giàu thông tin nhất	Vật hình khuyên, hình chữ L, hoặc bị che ở giữa — tâm rơi vào nền
ATSS	Ngưỡng $\mu + \sigma$ tính riêng cho từng vật từ k ứng viên gần nhất	Phân bố IoU của ứng viên là đơn đỉnh	Cảnh rất đông: các ứng viên “gần nhất” thuộc về vật khác
OTA / SimOTA	Vận tải tối ưu toàn ảnh trên chi phí phân loại + hồi quy	Chi phí ghép phản ánh đúng chất lượng dự đoán	Nhảy với chi phí khởi đầu khi mô hình còn kém; tốn thêm thời gian huấn luyện

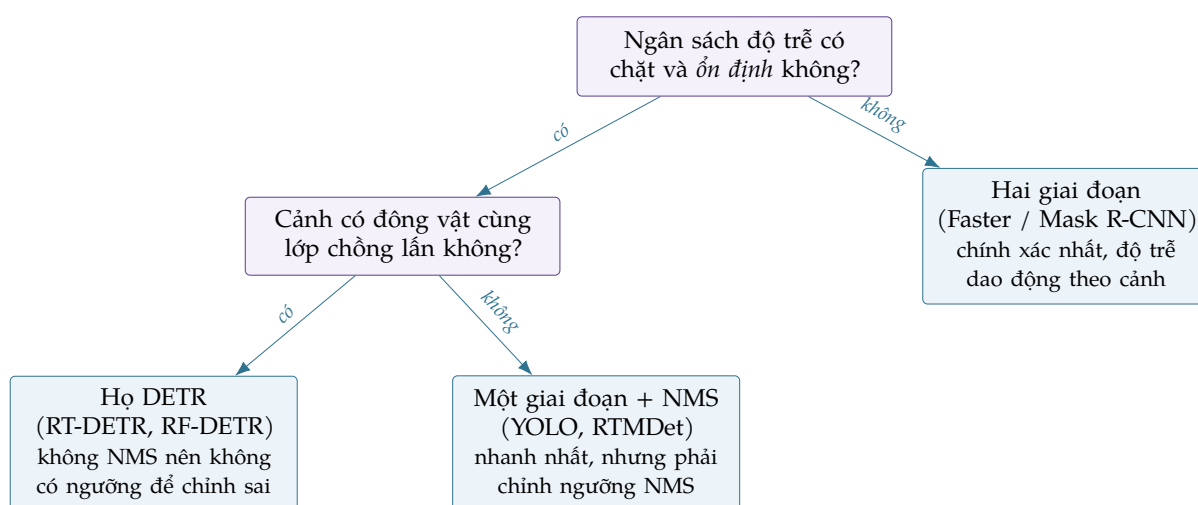
Bảng 1 nên đọc cùng với một quan sát khó chịu: bốn dòng này thay đổi *hàm mất mát*, không thay đổi mô hình. Hai detector khác nhau ở dòng gán mẫu là hai bài toán tối ưu khác nhau, nên so sánh chúng rồi quy kết chênh lệch cho “kiến trúc” là một suy luận sai ngay từ tiền đề.

↔ cùng nguyên lý với C/14/03 — Mask2Former mượn nguyên phép ghép cặp một-một này sang miền mask, và nhờ đó hợp nhất được semantic, instance và panoptic.

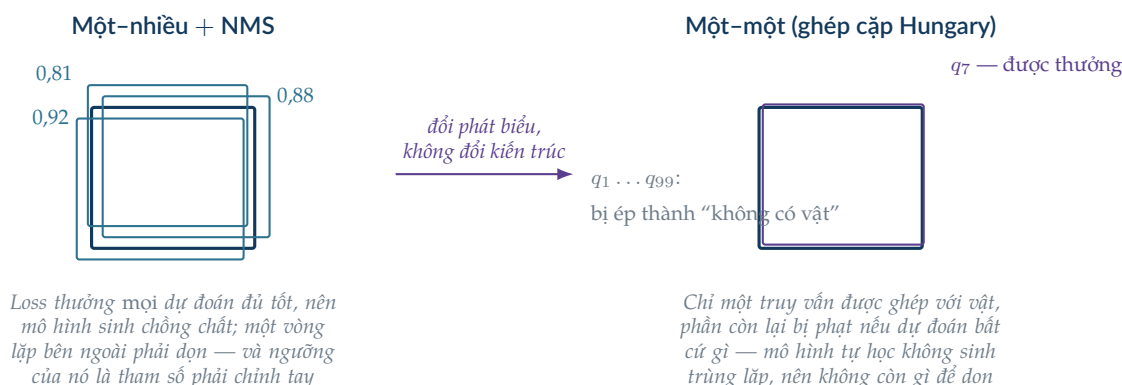
Hungarian matching của DETR, nhìn qua lăng kính này, chỉ là dòng thứ năm — một cách gán mẫu một-một thay vì một-nhiều — và chính điều đó, chứ không phải Transformer, mới là thứ khiến NMS trở nên thừa.

CẠM BÃY

Điểm số mà NMS dùng để xếp hạng là điểm *phân loại*, nhưng thứ ta cần giữ lại là khung *định vị* tốt nhất. Hai đại lượng này tương quan yếu hơn ta tưởng, nên một khung tự tin về nhân nhưng lệch vị trí vẫn có thể xoá mất một khung đúng hơn. Các bản vá là dự đoán thêm một nhánh ước lượng IoU hoặc centerness rồi nhân vào điểm; nhưng nếu bạn thấy mAP ở ngưỡng IoU chặt (AP_{75}) tụt nhiều hơn ở ngưỡng lỏng (AP_{50}), hãy nghi ngờ đúng chỗ này trước khi đổi backbone.



Hình 3. Cây quyết định thực dụng: ràng buộc loại bỏ phương án trước khi độ chính xác kịp lên tiếng. Hai nhánh trái đều là một giai đoạn, nhưng khác nhau ở chỗ NMS còn hay không — và trong cảnh đông đúc, đó là khác biệt quyết định chứ không phải vài điểm mAP.



Hình 4. Vì sao NMS biến mất khi đổi cách ghép. Bên trái, hàm mất mát chấm điểm từng dự đoán độc lập nên nhiều dự đoán cùng đúng và cùng được thưởng; việc chọn một trong số đó phải làm ở hậu xử lý, bằng một heuristic có ngưỡng. Bên phải, phép ghép một-một biến “sinh trùng lặp” thành hành vi bị phạt ngay trong hàm mất mát, nên bước dọn không còn lý do tồn tại. Các số tin cậy chỉ để minh họa.

Hình 4 cũng giải thích cái giá: vì mỗi vật chỉ có đúng một truy vấn được thưởng, tín hiệu giám sát cho mỗi truy vấn thừa hơn hẳn, và phép ghép còn dao động giữa các epoch — hai lý do khiến DETR bản gốc hội tụ chậm.

↪ **ràng buộc bởi B/03** — cây quyết định này là hiện thân của nguyên tắc đọc hệ thống: ràng buộc cứng loại phương án trước, độ chính xác chỉ lên tiếng sau.

Hình 3 tóm lại mục này dưới dạng dùng được ngay: câu hỏi đầu tiên không phải “mô hình nào mAP cao nhất” mà “ràng buộc nào của tôi là cứng” — đúng cách đọc hệ thống ở B/03-giai-phau-he-thong.

C. Giới hạn và trường hợp hồng

Detector một giai đoạn hồng ở ba chỗ đáng nhớ, mỗi chỗ có một triệu chứng quan sát được:

- **Nhãn nhiễu gặp focal loss.** Focal loss giả định nhãn sạch; trên dữ liệu gán bằng đám đông hoặc bán tự động, nó khuếch đại đúng những mẫu bị gán sai và có thể tệ hơn cross-entropy thường. *Triệu chứng:* loss huấn luyện đứng yên trong khi một nhóm rất nhỏ mẫu chiếm gần hết *gradient*.
- **Gán nhãn tinh vi ở giai đoạn đầu.** SimOTA và họ hàng dựa vào chi phí ghép, mà chi phí đó vô nghĩa khi mô hình còn kém, nên chúng phải kèm một giai đoạn khởi động. *Triệu chứng:* phân kỳ hoặc hội tụ về nghiệm suy biến trong vài epoch đầu — và chi tiết khắc phục nằm trong code chứ không trong bài.
- **Ngân sách truy vấn cố định của DETR.** Ảnh chứa nhiều vật hơn N thì phần dư bị mất im lặng, không có cảnh báo nào. *Triệu chứng:* recall giảm đều theo mật độ vật thể; không lộ ra trên COCO nhưng lộ ra ngay khi bạn đếm người trong đám đông.

Với người làm hệ thống thời gian thực, còn một giới hạn nữa ít được nói: bỏ NMS không có nghĩa là bỏ hậu xử lý không xác định. Một giai đoạn có thời gian chạy gần như cố định theo nội dung cảnh, đó là ưu điểm lớn nhất của nó so với hai giai đoạn; nhưng nếu bạn thêm lại NMS trên đầu ra (như hầu hết bản triển khai YOLO làm), thời gian NMS lại tỉ lệ với số khung sống sót, và phương sai độ trễ quay về.

CẦN NHỚ

Trước khi tin bất kỳ so sánh detector nào, hãy hỏi hai câu: hai bên dùng cách gán mẫu nào, và hai bên chạy bao nhiêu epoch với augmentation gì. Rất nhiều “cải tiến kiến trúc” công bố thực chất là cải tiến gán mẫu, và nó bị chôn trong code. [T3]

D. Kết nối

C/14-bai-toan-cv-loi/01-detection-hai-giai-doan là tiền đề trực tiếp: mất cân bằng chỉ lộ ra khi bỏ giai đoạn đề xuất, nên mục này chỉ có nghĩa sau mục kia. B/09-thiet-ke-ham-mat-mat là nơi focal loss được đặt cạnh các cách xử lý mất cân bằng khác, và là nơi trả lời câu hỏi tổng quát “khi nào nên sửa ở loss thay vì sửa ở dữ liệu”. Ý tưởng dự đoán tập hợp bằng Hungarian matching quay lại nguyên vẹn ở C/14-bai-toan-cv-loi/03-segmentation dưới dạng khung truy vấn của Mask2Former — đó là trường hợp hiếm hoi một ý tưởng chuyển miền mà không mất gì. E/24-thi-nghiem-va-ablation cung cấp phương pháp để tự kiểm chứng khẳng định trung tâm của mục này.

❗ **Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Vì sao mất cân bằng foreground–background hầu như không xuất hiện trong Faster R-CNN nhưng lại nghiêm trọng trong SSD, dù cả hai dùng cùng anchor?
2. Hệ số điều biến với $\gamma = 2$ bằng bao nhiêu tại $p_t = 0,9$ và tại $p_t = 0,3$? Hai con số đó giải thích thế nào việc focal loss nguy hiểm trên dữ liệu nhiễu?
3. ATSS cho thấy điều gì về câu hỏi “anchor hay anchor-free”? Kết quả đó thay đổi cách bạn đọc một paper detection như thế nào?
4. DETR bỏ được NMS nhờ Hungarian matching chứ không nhờ Transformer. Hãy giải thích tại sao, và điều gì sẽ xảy ra nếu ta ghép một–nhiều thay vì một–một?
5. Bạn đo được AP_{50} cao nhưng AP_{75} thấp — hai giả thuyết nào giải thích được, và thí nghiệm nào phân biệt chúng?
6. Trên một robot có ngân sách 20 ms mỗi khung hình, vì sao phương sai độ trễ có thể quan trọng hơn độ trễ trung bình?

Nguồn

- [1] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [2] W. Liu, D. Anguelov, D. Erhan, et al., “SSD: Single Shot MultiBox Detector,” in *ECCV*, 2016.
- [3] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [4] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [5] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [6] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [7] Z. Ge, S. Liu, Z. Li, et al., “OTA: Optimal Transport Assignment for Object Detection,” in *CVPR*, 2021.
- [8] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [9] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [10] X. Zhu, W. Su, L. Lu, et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *ICLR*, 2021.
- [11] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [12] H. Zhang, F. Li, S. Liu, et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” in *ICLR*, 2023.